

The Shell (Part 2)

NCSC101

Prof. Pranay Kumar Saha

August 06, 2025





Viewing File Contents

cat - Concatenate and Display

Prints the entire content of a file to the screen.

```
$ cat config.txt
```

less - View File Page by Page

Lets you scroll through large files. More powerful than **more**.

```
$ less /var/log/syslog
```

(Press **q** to quit, arrow keys to navigate).



Comparison: more vs. less command

Feature	more	less
Navigation	Historically forward-only; modern implementations support limited backward movement: page-wise (b , Ctrl-B); line-wise (k) on some systems.	Full bidirectional navigation: line- and page-wise moves (arrow keys, Space , b).
Search	No interactive search within pager (except initial pattern with more +/pattern).	Forward (/pattern) and backward (?pattern) search with n/N .
Jump to start/end	Not supported.	Jump directly to start (g) or end (G) of file.



Viewing File Contents(contd...)

head / tail

View the beginning or end of a file.

```
$ head -n 5 file.txt # First 5 lines  
$ tail -n 10 file.txt # Last 10 lines
```



Getting Help

man - The Manual

The most important command! It shows you the manual page for any other command.

```
$ man ls
```

This will open a detailed document explaining what **ls** does and all its available options.

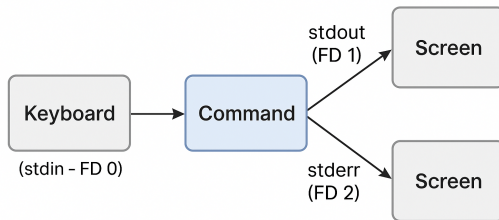
- Navigate **man** pages just like **less**.
- Press **q** to exit.
- When in doubt, **man** it out!



Standard Streams: `stdin`, `stdout`, `stderr`

Every command has three standard data streams associated with it.

- **Standard Input (`stdin`):** Where the command gets its input from. By default, this is your keyboard. (File Descriptor 0)
- **Standard Output (`stdout`):** Where the command sends its normal output. By default, this is your screen. (File Descriptor 1)
- **Standard Error (`stderr`):** Where the command sends its error messages. By default, this is also your screen. (File Descriptor 2)





Standard Stream

Three streams: stdin (0), stdout (1), stderr (2).

- `>` redirect stdout (overwrite).
- `>>` append stdout.
- `<` redirect stdin.
- `2>` redirect stderr.



I/O Redirection: Changing the Flow

You can redirect these streams to or from files.

Redirecting stdout (>)

Sends normal output to a file instead of the screen. **Overwrites the file.**

```
$ ls -l > file_list .txt
```

Redirect Errors

```
$ ls -l 2> file_list .txt
```

Appending stdout (>>)

Sends normal output to a file. **Appends to the end of the file.**

```
$ echo "New log entry" >> application.log
```




I/O Redirection: Changing the Flow

Redirecting stdin (<)

Takes input from a file instead of the keyboard.

```
$ sort < names.txt
```



Pipes: Chaining Commands

The Pipe Operator (|)

A pipe is one of the most powerful features. It sends the **stdout** of the command on the left to the **stdin** of the command on the right.

Command A (stdout) -> (stdin) Command B

Example: Find all .txt files

- **ls -l**: Produces a long list of all files.
- **grep ".txt"**: Searches its input for lines containing ".txt".

```
$ ls -l | grep ".txt"
-rw-r--r-- 1 user user 45 Jul 29 11:25 file1.txt
-rw-r--r-- 1 user user 89 Jul 29 11:40 notes.txt
```



Regular Expressions in Unix Commands: RegEx

Regular expressions (REs) are concise patterns for matching text. In Unix, they are most commonly used with commands like **grep**, **sed**, and **awk**.

Core Metacharacters with Examples

- **.** —matches any single character (except newline).
*Example: **c.t** matches "cat" , "cot" , "c9t" , etc.*



Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- **^, \$** —anchors start and end of line.

Examples:

^Hello matches “Hello” at line start.

end\$ matches “friend” or “the end” at line end.

- ***, +, ?** —quantifiers:

***** = zero-or-more, **+** = one-or-more, **?** = zero-or-one of preceding atom.

Examples:

go*d matches “gd” , “god” , “goood” , ...

ab+ matches “ab” , “abb” , “abbb” , ...

colou?r matches “color” or “colour” .



Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- **{m,n}** —match at least *m*, at most *n* occurrences of preceding atom.

Example:

[0-9]{2,4} matches “12” , “123” , or “1234” but not “1” or “12345” .

- **[abc]**, **[a-z0-9]** —character classes: match exactly one of the listed characters or ranges.

Examples:

gr[ae]y matches “gray” or “grey” .

[A-Za-z0-9] matches any single alphanumeric.

- **[^...]** —negated class: match any character *_not_* listed.

Example: **[^0-9]** matches any non-digit character.



Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- `(...)` —grouping: treat the subpattern as a unit.
Example: `(cat|dog)s?` matches “cat” , “cats” , “dog” , or “dogs” .
- `|` —alternation (logical OR, in **grep -E**).
Example: `red|blue|green` matches any of “red” , “blue” , or “green” .
- `\` —escape the next character to match it literally.
Example: `\.` matches a literal dot “.” instead of “any character” .



grep

grep (Global Regular Expression Print) searches text for patterns and outputs matching lines. Common options:

- **-i**: Case-insensitive search.
- **-v**: Invert match (select non-matching lines).
- **-c**: Count matching lines.
- **-n**: Prefix each line with its line number.
- **-w**: Match whole words only.
- **-r**: Recursive search through directories.
- **-E**: Use Extended Regular Expressions.
- **-o**: Show only the matching part of the line.



Searching Text with grep

grep searches for patterns in text. It's one of the most used commands.

Common Uses

Find lines containing "error" in a log file

```
$ grep "error" /var/log/syslog
```

Search case-insensitively for "user"

```
$ grep -i "user" access.log
```

Recursively search for "config" in a directory

```
$ grep -r "config" /etc/
```

Show lines that DO NOT match a pattern

```
$ grep -v "success" results.txt
```




Finding Files: find and locate

Two different tools for finding files by name.

find

- Searches the filesystem in real-time.
- Powerful and flexible.
- Can be slower.

```
# Find files by name
```

```
$ find /home -name "*.pdf"
```

```
# Find directories
```

```
$ find . -type d -name "images"
```

locate

- Searches a pre-built database.
- Extremely fast.
- May not find recently created files.

```
# locate is simple
```

```
$ locate my_report.docx
```

```
# Admin can update db
```

```
$ sudo updatedb
```



Counting and Sorting: sort, uniq, wc

These three utilities are often used together to process lists of text.

wc - Word Count

Counts lines, words, and characters.

```
$ wc -l user_list.txt # Count only lines
```

Flag	Description
------	-------------

- | | |
|----|--|
| -l | Count lines (newline characters). |
| -w | Count words (whitespace-separated). |
| -c | Count bytes. |
| -m | Count characters (in multibyte locales). |
| -L | Print length of the longest line. |



Advanced Processing: sed and awk

These are powerful tools for complex text manipulation, often used in scripts.

sed - Stream Editor

Performs text transformations, like search-and-replace.

```
# Replace all instances of 'old' with 'new'  
$ sed 's/old_text/new_text/g' input.txt
```

awk - A Pattern-Scanning Language

Excellent for processing files with columns of data.

```
# Print the first and ninth columns from ls -l  
$ ls -l | awk '{print $1, $9}'
```



The Shell (Part 2)

Thank You for Listening!